

UNIVERSIDAD EL BOSQUE

Ingeniería de Sistemas
Bases de Datos I

DOCUMENTO DE LA IMPLEMENTACIÓN DE LA VM

Angela Maria Reina Vivas
Angel Felipe Alvarez Benavides
Cristian David Vargas Cetina

Asignatura: Bases de Datos I
Docente: Christian Felipe Duarte Arevalo

Bogotá D.C.
2026

1. Introducción

Este documento describe el proceso completo de implementación del sistema de gestión para gimnasio sobre una máquina virtual. El objetivo era desplegar tanto el backend como el frontend en un entorno virtualizado, de modo que el sistema fuera accesible dentro de la red local sin depender directamente de las máquinas de los desarrolladores.

Cabe aclarar desde el inicio que este documento narra no solo el proceso exitoso de despliegue del backend, sino también los intentos realizados para integrar el frontend en la máquina virtual y las razones por las cuales, luego de múltiples pruebas, se tomó la decisión de mantener únicamente el servidor backend corriendo en la VM. Esta decisión fue de carácter técnico y práctico, y se explica en detalle en la sección correspondiente.

Alcance del despliegue

Componentes desplegados en la VM:

- API REST (Spring Boot) en el puerto 8080 → EXITOSO
- Base de datos PostgreSQL (en máquina host, accesible desde la VM) → EXITOSO
- Frontend (JavaScript / Nginx) → INTENTADO — No implementado en producción

2. Entorno de Virtualización

2.1 Hipervisor seleccionado: Oracle VM VirtualBox

Para este proyecto académico se escogió Oracle VM VirtualBox como hipervisor. Esta decisión estuvo motivada principalmente por el contexto educativo y la disponibilidad de recursos. A continuación se describen los motivos principales de esta elección:

- Gratuito y de código abierto: VirtualBox no tiene costo de licencia, lo que lo hace ideal para proyectos académicos donde no se cuenta con presupuesto para software de virtualización.
- Multiplataforma: Funciona sobre Windows, macOS y Linux, lo que permitió que cualquier integrante del equipo pudiera utilizarlo independientemente del sistema operativo de su computador personal.
- Facilidad de uso: La interfaz gráfica de VirtualBox es intuitiva y permite configurar la red, los recursos y los snapshots sin necesidad de conocimientos avanzados en administración de sistemas.
- Soporte de red en modo bridge: VirtualBox permite configurar la red en modo puente (Bridged Adapter), lo que asigna a la VM una dirección IP dentro de la red local real. Esto fue fundamental para que el backend en la VM fuera accesible desde el computador host y desde el frontend.
- Amplia documentación y comunidad: Al ser uno de los hipervisores más utilizados en entornos educativos, existe abundante documentación, tutoriales y soluciones a errores comunes en foros y comunidades en línea.

Configuración de red utilizada

Adaptador de red: Bridged Adapter (Adaptador puente)

- IP asignada a la VM: 192.168.1.50
- IP del host (equipo físico con PostgreSQL): 192.168.1.14 (en casa) / 192.168.1.57 (en red externa)
- Razón del modo bridge: permite que la VM tenga una IP en la red local, haciendo el backend accesible desde cualquier dispositivo de la red sin necesidad de reenvío de puertos.

2.2 Sistema operativo: Debian GNU/Linux 13 (Trixie)

Como sistema operativo invitado se eligió Debian GNU/Linux 13, conocido por su estabilidad, seguridad y la calidad de su sistema de gestión de paquetes APT. En un entorno académico, Debian es una excelente elección porque:

- Es completamente gratuito y de distribución libre.
- Sus repositorios incluyen paquetes actualizados de Java, Maven y PostgreSQL-client, necesarios para el proyecto.

- Tiene un consumo de recursos bajo, lo que permite correr la VM sin afectar significativamente el rendimiento del equipo host.
- Es ampliamente utilizado como sistema base en servidores de producción, por lo que el conocimiento adquirido es directamente aplicable a entornos reales.

Parámetro	Valor configurado
Sistema operativo	Debian GNU/Linux 13 (Trixie) — 64 bits
RAM asignada	2048 MB (2 GB)
Almacenamiento	20 GB (disco virtual VDI)
Adaptador de red	Bridged Adapter (modo puente)
IP de la VM	192.168.1.50
Usuario principal	angela
Versión de Java	OpenJDK 21.0.11
Herramienta de build	Apache Maven

3. Instalación y Configuración del Backend

3.1 Transferencia del proyecto a la VM

El proyecto backend fue desarrollado localmente y versionado en un repositorio Git. Para transferirlo a la VM se utilizó el protocolo SCP, que permite copiar archivos entre equipos a través de SSH de manera segura.

El proyecto se encontraba en la siguiente ruta del equipo Windows:

```
C:\Users\angel\Downloads\ProyectoBD\
```

Una vez identificada la ruta del proyecto en la VM mediante comandos de exploración, se encontró la estructura del backend en:

```
/home/angela/Gym/Gym/
```

Dentro de esta carpeta se confirmó la presencia de los archivos esenciales del proyecto Maven:

- pom.xml — archivo de configuración del proyecto y sus dependencias
- src/ — código fuente con los paquetes del backend
- mvnw — wrapper de Maven para compilación
- target/ — directorio generado tras la compilación

3.2 Configuración del application.properties

El archivo de propiedades de Spring Boot debió ajustarse para que el backend se conectara correctamente a la base de datos PostgreSQL que se encontraba en el equipo host (máquina Windows). La configuración final utilizada fue:

```
spring.application.name=Gym
spring.datasource.url=jdbc:postgresql://192.168.1.14:5432/gym
spring.datasource.username=postgres
spring.datasource.password=Data123
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.orm.jdbc.bind=trace
```

Un detalle importante es que la IP del host cambiaba dependiendo de la red a la que se conectara el equipo. Cuando el equipo estuvo en una red diferente a la del hogar, la IP cambió de 192.168.1.14 a 192.168.1.57, lo que requirió actualizar manualmente el archivo de propiedades y recompilar el proyecto. Esto evidenció una de las limitaciones del enfoque con base de datos en el host físico en lugar de instalarla directamente en la VM.

3.3 Proceso de compilación

La compilación del proyecto presentó varios inconvenientes que fue necesario resolver antes de obtener el archivo JAR ejecutable. A continuación se documentan los pasos seguidos y los problemas encontrados:

3.3.1 Problema con el archivo mvnw

Al intentar ejecutar el wrapper de Maven, el sistema reportó permiso denegado:

```
./mvnw clean package -DskipTests
# Error: Permission denied
```

Solución aplicada:

```
chmod +x /home/angela/Gym/Gym/mvnw
```

3.3.2 Problema con la clase Main no encontrada

Maven no reconocía automáticamente la clase principal del proyecto. Esto se debió a que el pom.xml no tenía configurado el plugin de Spring Boot con la clase main explícita. Adicionalmente, existían dos archivos GymApplication.java en el proyecto: uno en el paquete com.example (incompleto) y el correcto en el paquete co.edu.unbosque.

Se identificó la clase correcta mediante:

```
find /home/angela/Gym/Gym/src -name '*Application*.java'
# Resultado: /home/angela/Gym/Gym/src/main/java/co/GymApplication.java
# Paquete: co.edu.unbosque
```

Se editó el pom.xml para agregar la configuración del plugin:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <mainClass>co.edu.unbosque.GymApplication</mainClass>
      </configuration>
    </plugin>
  </plugins>
</build>
```

3.3.3 JAR corrupto (aparecía en rojo en el sistema de archivos)

Después de la primera compilación exitosa, al listar el directorio target se observó que el archivo JAR aparecía resaltado en rojo en la terminal Linux. Esto indicaba que era un enlace simbólico roto, probablemente originado por una transferencia incorrecta del proyecto. Cada vez que se intentaba lanzar el JAR rojo, el sistema reportaba:

```
java -jar target/Gym-0.0.1-SNAPSHOT.jar  
# Error: unable to access jarfile
```

La solución fue eliminar el directorio target y recompilar usando el comando completo:

```
cd /home/angela/Gym/Gym  
rm -rf target  
mvn clean compile package -DskipTests
```

Compilación exitosa

Tras ejecutar el comando anterior, el BUILD SUCCESS fue confirmado y el archivo JAR se generó correctamente en target/Gym-0.0.1-SNAPSHOT.jar. El archivo ya no aparecía en rojo.

4. Configuración de Conectividad con PostgreSQL

Dado que PostgreSQL estaba instalado en el equipo host Windows y no directamente en la VM, fue necesario configurar PostgreSQL para aceptar conexiones remotas provenientes de la dirección IP de la VM (192.168.1.50). Este proceso implicó modificar dos archivos de configuración de PostgreSQL y configurar el firewall de Windows.

4.1 Modificación de postgresql.conf

Por defecto, PostgreSQL solo escucha conexiones provenientes de localhost. Para permitir conexiones desde la VM fue necesario cambiar el parámetro `listen_addresses` en el archivo de configuración principal. El archivo se ubicaba en:

C:\Program Files\PostgreSQL\18\data\postgresql.conf

Cambio realizado:

Antes:

`listen_addresses = 'localhost'`

Después:

`listen_addresses = '*'`

4.2 Modificación de pg_hba.conf

El archivo `pg_hba.conf` controla qué clientes pueden conectarse a PostgreSQL y con qué método de autenticación. Se agregó una línea al final del archivo para permitir la conexión desde la IP de la VM:

C:\Program Files\PostgreSQL\18\data\pg_hba.conf

Configuración final que funcionó:

Línea agregada al final de pg_hba.conf:

`host all all 0.0.0.0/0 trust`

Adicionalmente, fue necesario forzar la recarga de la configuración desde pgAdmin mediante:

```
SELECT pg_reload_conf();
```

4.3 Configuración del firewall de Windows

Windows Defender bloqueaba las conexiones entrantes al puerto 5432. Fue necesario crear reglas de firewall tanto para ICMP (ping) como para el puerto de PostgreSQL. Los comandos ejecutados en PowerShell como administrador fueron:

```
New-NetFirewallRule -DisplayName "ICMP Allow" -Protocol ICMPv4 -IcmpType 8 -Direction Inbound -Action Allow
```

```
New-NetFirewallRule -DisplayName "PostgreSQL 5432" -Direction Inbound -Protocol TCP -LocalPort 5432 -Action Allow
```

```
New-NetFirewallRule -DisplayName "Puerto 8080" -Direction Inbound -Protocol TCP -LocalPort 8080 -Action Allow
```

Una dificultad adicional fue que el primer intento de ejecutar este comando arrojó 'Acceso denegado', lo que indicaba que PowerShell no se había abierto con privilegios de administrador. Fue necesario reabrirlo con la opción 'Ejecutar como administrador'.

4.4 Verificación de conectividad

Para confirmar que la VM podía alcanzar el servidor PostgreSQL, se instaló el cliente de PostgreSQL en la VM:

```
su root  
apt install postgresql-client -y
```

Y se verificó la conexión:

```
psql -h 192.168.1.14 -U postgres -d gym  
# Resultado exitoso: conexión establecida
```

5. Lanzamiento del Backend en la VM

Una vez resueltos los problemas de compilación y conectividad con la base de datos, el backend pudo lanzarse exitosamente mediante el siguiente comando ejecutado en la VM:

```
java -jar /home/angela/Gym/Gym/target/Gym-0.0.1-SNAPSHOT.jar
```

Para verificar que el backend estaba escuchando correctamente en el puerto 8080, se utilizó el siguiente comando:

```
ss -tlnp | grep 8080  
# Resultado: 0.0.0.0:8080 — confirmado
```

5.1 Consideración sobre redes externas

Durante el desarrollo del proyecto se presentó una situación interesante: al cambiar de red (por ejemplo, al conectar el equipo desde una ubicación diferente), la IP del host cambiaba. Esto hacía que el backend en la VM no pudiera conectarse a PostgreSQL porque la dirección configurada en `application.properties` (192.168.1.14) ya no era válida.

La solución adoptada fue identificar la nueva IP del host con `ipconfig`, actualizar el `application.properties` en la VM, recompilar y relanzar el backend. Esta limitación refuerza la necesidad de tener la base de datos instalada directamente en la VM en un entorno más robusto, eliminando la dependencia de la IP del host físico.

Procedimiento para relanzar el sistema tras un cambio de red

1. Verificar la nueva IP del host: `ipconfig` (Windows)
2. Actualizar `application.properties` en la VM con la nueva IP
3. Recompilar: `mvn clean compile package -DskipTests`
4. Relanzar: `java -jar target/Gym-0.0.1-SNAPSHOT.jar`
5. Verificar acceso: `http://192.168.1.50:8080/api/clientes` desde el navegador

6. Intentos de Despliegue del Frontend en la VM

El frontend del proyecto fue desarrollado en JavaScript, organizado en módulos ES6 con archivos View separados por entidad (ClienteView.js, InstructoresView.js, SesiónView.js, etc.). Durante el desarrollo, el frontend se ejecutaba localmente desde VS Code con la extensión Live Server, conectándose al backend mediante el ApiService configurado con la IP de la VM.

Sin embargo, se intentó también desplegar el frontend directamente en la VM utilizando Nginx como servidor web estático, con el objetivo de que toda la aplicación (frontend + backend) corriera desde la VM sin depender del computador del desarrollador.

6.1 Transferencia del frontend a la VM

El frontend se encontraba en la misma carpeta del proyecto en Windows. Se transfirió a la VM utilizando SCP desde PowerShell:

```
scp -r C:\Users\angel\Downloads\ProyectoBD\gym-app angela@192.168.1.50:/home/angela/
```

Al ejecutar el comando, la VM solicitó confirmación de la clave del host SSH:

```
The authenticity of host '192.168.1.50' can't be established.  
ED25519 key fingerprint is SHA256:4jypQePfyulvjHWxu234yNFJkLYbRyN+O2cckgCHz+U.  
# Se respondió 'yes' y la transferencia se completó exitosamente
```

6.2 Instalación y configuración de Nginx

Se procedió a instalar Nginx en la VM para servir los archivos estáticos del frontend:

```
The authenticity of host '192.168.1.50' can't be established.  
ED25519 key fingerprint is SHA256:4jypQePfyulvjHWxu234yNFJkLYbRyN+O2cckgCHz+U.  
# Se respondió 'yes' y la transferencia se completó exitosamente
```

Al acceder a <http://192.168.1.50> desde el navegador, la página principal del frontend se cargó correctamente. Sin embargo, al intentar navegar entre las secciones de la aplicación, ningún botón respondía.

6.3 Error de MIME type en módulos JavaScript

Al inspeccionar la consola del navegador (F12), se identificaron múltiples errores en rojo que indicaban el problema real:

```
Failed to load module script: Expected a JavaScript module script but the server responded with a MIME type of "text/html". Strict MIME type checking is enforced  
}
```

El error se debía a que Nginx no estaba sirviendo los archivos .js con el tipo MIME correcto (application/javascript). En lugar de eso, los estaba sirviendo como text/html, lo que hacía que el navegador rechazara cargarlos como módulos ES6.

6.4 Intentos de corrección de la configuración de Nginx

Se realizaron múltiples intentos para corregir la configuración de Nginx y que sirviera los archivos JavaScript con el tipo MIME correcto:

Intento 1: Configuración con tipos MIME explícitos y bloques location

```
server {
    listen 80 default_server;
    root /var/www/html;
    index index.html;
    server_name _;
    location / {
        try_files $uri $uri/ /index.html;
    }
    location ~* \.js$ {
        add_header Content-Type application/javascript;
        try_files $uri =404;
    }
    location ~* \.css$ {
        add_header Content-Type text/css;
        try_files $uri =404;
    }
}
```

Intento 2: Uso de directiva include mime.types

Si bien esta configuración es técnicamente correcta, el problema persistía porque los archivos JS del proyecto usaban import/export de módulos ES6 (type='module' en el HTML). Nginx requería configuración adicional específica para manejar este tipo de módulos correctamente con los encabezados adecuados.

Intento 3: Reescritura completa del archivo de configuración

Se intentaron múltiples veces sobrescribir el archivo de configuración usando heredoc (cat > archivo << 'EOF'...EOF) para evitar problemas de escritura manual, pero el archivo anterior no se borraba correctamente y quedaban dos bloques server {} en conflicto, generando errores de sintaxis que impedían que Nginx iniciara.

```
# Error al verificar sintaxis:
/usr/sbin/nginx -t
# nginx: [emerg] "server_name" directive is not allowed here
```

```
# nginx: configuration file /etc/nginx/nginx.conf test failed
```

Intento 4: Edición manual en nano

Se intentó editar el archivo directamente en nano, borrando todo el contenido con Ctrl+K línea por línea y reescribiendo la configuración. Sin embargo, las limitaciones del teclado en la interfaz de VirtualBox y la dificultad para ingresar ciertos caracteres especiales (como ~, @, \) hicieron que el proceso fuera propenso a errores.

Eventualmente se logró que Nginx iniciara correctamente (active running), pero el problema del MIME type en los módulos JavaScript persistía porque la configuración simplificada sin los bloques location específicos para .js no era suficiente para los módulos ES6.

7. Decisión de Mantener Solo el Backend en la VM

Después de múltiples intentos de configurar el frontend en Nginx, se tomó la decisión de mantener únicamente el backend (API REST de Spring Boot) corriendo en la VM, y dejar el frontend ejecutándose localmente desde VS Code con Live Server.

7.1 Razones técnicas

Los problemas encontrados al intentar desplegar el frontend en Nginx estuvieron relacionados principalmente con:

6. La incompatibilidad entre la forma en que Nginx servía los archivos JavaScript y los requerimientos de los módulos ES6. El frontend usaba import/export nativos del navegador, lo que exige que el servidor responda con el header Content-Type correcto (application/javascript) y que el HTML tenga type='module' en sus scripts.
7. La existencia de múltiples bloques server{} en el archivo de configuración de Nginx que generaban conflictos de sintaxis y hacían fallar el servicio.
8. El tiempo requerido para depurar la configuración de Nginx superaba el valor agregado de tener el frontend en la VM para este contexto académico.

7.2 Funcionamiento final del sistema

La arquitectura final que quedó operativa fue la siguiente:

Componente	Ubicación	Acceso
Frontend (JS + HTML)	Máquina host — VS Code + Live Server	http://localhost:[puerto]
Backend (Spring Boot)	VM Debian — puerto 8080	http://192.168.1.50:8080
Base de datos (PostgreSQL)	Máquina host — puerto 5432	localhost:5432 (desde el host)

Con esta configuración, el frontend en el host hace peticiones HTTP al backend en la VM, y el backend se conecta a PostgreSQL en el mismo host por la red local. El sistema funcionó correctamente para todas las operaciones CRUD sobre las entidades del gimnasio.

8. Verificación Final del Sistema

8.1 Pruebas de conectividad

Prueba	Comando / URL	Resultado
Ping de VM a host	ping 192.168.1.14	64 bytes recibidos — Exitoso
Conexión a PostgreSQL desde VM	psql -h 192.168.1.14 -U postgres -d gym	Conexión establecida — Exitoso
Backend respondiendo	http://192.168.1.50:8080/api/clientes	JSON con lista de clientes — Exitoso
Backend escuchando en 8080	ss -tlnp grep 8080	0.0.0.0:8080 confirmado — Exitoso
Frontend consumiendo API	Live Server → peticiones a VM	CRUD funcional — Exitoso

8.2 Procedimiento para iniciar el sistema

Para volver a lanzar el sistema completo después de apagar los equipos, los pasos son los siguientes:

9. Verificar que PostgreSQL esté corriendo en Windows: Servicios → postgresql-x64-18 → Estado: Running.
10. Iniciar la VM en VirtualBox y verificar que tenga la IP 192.168.1.50 con ip a.
11. Lanzar el backend en la VM:

```
java -jar /home/angela/Gym/Gym/target/Gym-0.0.1-SNAPSHOT.jar
```

12. Verificar que el backend responda: abrir <http://192.168.1.50:8080/api/clientes> en el navegador.
13. Abrir el frontend en VS Code y lanzar con Live Server.

9. Reflexión Académica

Este proceso de despliegue fue una experiencia de aprendizaje muy completa, aunque también bastante desafiante. Al principio, pensábamos que iba a ser relativamente sencillo: compilar el proyecto, copiarlo a la VM y lanzarlo. La realidad fue que cada paso tuvo sus propios obstáculos.

Los problemas con los permisos del archivo mvnw, el JAR corrupto que aparecía en rojo, la configuración de PostgreSQL para aceptar conexiones remotas, los ajustes del firewall de Windows: cada uno de estos fue un problema que nunca habíamos enfrentado antes y que requirió investigar, probar y equivocarse varias veces antes de encontrar la solución correcta.

El intento de desplegar el frontend en Nginx fue particularmente frustrante porque llegamos muy cerca de lograrlo. El problema del MIME type en los módulos ES6 es algo que, en retrospectiva, tiene una solución técnica clara: configurar correctamente los bloques location en Nginx y asegurarse de que el HTML incluya type='module'. Sin embargo, las limitaciones prácticas del teclado en la interfaz de VirtualBox y el tiempo disponible nos llevaron a tomar la decisión pragmática de mantener el frontend local.

Lo que más nos llevamos de esta experiencia es la comprensión de cómo se comunican los diferentes componentes de una aplicación distribuida: el frontend haciendo peticiones HTTP a una IP y puerto específicos, el backend procesando esas peticiones y consultando una base de datos en otro servidor, y todos los elementos de red y seguridad (firewall, configuración de acceso remoto en PostgreSQL, modo bridge en VirtualBox) que deben estar correctamente configurados para que todo funcione.

10. Conclusiones

El proyecto de implementación de la máquina virtual logró su objetivo principal: tener el backend de la aplicación GYM corriendo en un entorno de servidor virtualizado, separado del entorno de desarrollo local. El sistema es funcional y permite realizar todas las operaciones CRUD sobre las entidades de la base de datos a través de la API REST expuesta en el puerto 8080 de la VM.

El proceso evidenció que el despliegue de aplicaciones en entornos virtualizados involucra muchos más aspectos de los que inicialmente se anticipan: configuración de red, permisos del sistema de archivos, métodos de autenticación de bases de datos, reglas de firewall, tipos MIME en servidores web, entre otros. Cada uno de estos aspectos requirió investigación y solución de problemas en tiempo real.

La decisión de mantener únicamente el backend en la VM fue pragmática y válida para el contexto académico del proyecto. El frontend, al ser archivos estáticos JavaScript, puede ejecutarse desde cualquier entorno con acceso a la red y apuntar al backend en la VM sin ningún problema. Lo importante, desde el punto de vista de la materia, era demostrar la capacidad de desplegar la capa de persistencia y la lógica de negocio en un entorno de servidor separado.

Objetivo	Estado
VM configurada con Debian 13 en VirtualBox	Completado
Backend Spring Boot corriendo en la VM	Completado
Conexión exitosa con PostgreSQL en el host	Completado
API REST accesible desde la red local	Completado
Frontend desplegado en Nginx en la VM	No implementado — decisión de proyecto
Sistema completo funcional (front en host + back en VM)	Completado